



UNIVERSIDAD CATÓLICA DE SALTA
Licenciatura en Ciencia de Datos

Trabajo Práctico N° 1

Programación I

Alumno: Facundo Luna

Profesor: Ing. Mario Martínez

Fecha: 22 de marzo de 2026

Índice

Introducción	3
1. Lenguajes de programación más demandados	4
1.1. Metodología de investigación	4
1.2. Los 5 lenguajes más demandados en Argentina	4
1.2.1. 1. Python	4
1.2.2. 2. JavaScript	4
1.2.3. 3. Java	5
1.2.4. 4. C# (C Sharp)	5
1.2.5. 5. SQL (Structured Query Language)	5
1.3. Comparación de niveles de abstracción	6
1.4. Apreciación personal: El futuro de la programación	6
2. ¿Qué es una computadora?	8
3. ¿Qué es la lógica? Ejemplos de la vida real donde aplique la lógica	9
4. ¿Un Programa tiene relación con la lógica? Justificar	10
5. ¿Qué es un lenguaje? Dar ejemplos reales	11
6. ¿Cómo se relaciona la matemática con la informática?	12
7. ¿Qué operadores aritméticos y lógicos conoce?	13
7.1. Operadores Aritméticos	13
7.2. Operadores Lógicos	13
7.3. Operadores Relacionales	14
8. ¿Sabe de qué hablamos con la jerarquía de las operaciones?	15
8.1. Reglas de jerarquía	15
8.2. Regla de asociatividad	15
9. ¿Cómo se puede alterar la jerarquía natural de las operaciones?	16

10.¿Qué es un algoritmo?	17
11.¿En dónde encontramos algoritmos en la vida real?	18
12.Realizar el algoritmo de despertar y levantarse diariamente	19
12.1. Descripción del algoritmo	19
13.¿Qué es un programa? ¿Qué es un lenguaje de programación? ¿Y qué tarea es la que hace un programador?	21
14.¿A qué se denomina un programador Full-Stack?	22
15.¿Qué fases en la “resolución de problemas” conoce? Mencionelas, seleccione la que le parezca más le llame la atención explicando su importancia y sumando un ejemplo	23
15.1. Fases de la resolución de problemas	23
15.2. Fase que más me interesa: Análisis y Diseño	23
15.3. Ejemplo práctico	24
16.¿Qué es una variable y qué es una constante?	26
17.¿Qué es Software y qué es Hardware?	27
18.¿Qué es la memoria RAM y la memoria ROM?	28
19.¿Qué son los dispositivos de entrada y salida? Dé 5 ejemplos de cada uno	29
20.Nombrar las unidades principales de un procesador	30

Introducción

El actual trabajo práctico tiene preguntas introductorias conceptuales a la materia, algunas a modo diagnóstico, otras a modo nivelación y otras ya específicos del contenido de la unidad 1 de la materia. También podrá encontrar respuestas a ciertas consultas en las unidades posteriores del módulo, con la intención de que vaya avanzando en la lectura de estos.

Puede consultarse el material de estudio y también investigar otras fuentes de información a su gusto, como ser sitios de internet, publicaciones, bibliografía deseada, etc.

Todos los trabajos prácticos son de presentación **individual**. Presentar el trabajo en un documento Word o Pdf. Se valorará (en todos los prácticos) la prolijidad, el contenido, el trabajo investigativo, las opiniones/aportes personales y la presentación en tiempo y forma según la fecha de entrega, lo que con el correr de los TP derivará en la posibilidad de regularizar y/o promocionar la materia.

1. Lenguajes de programación más demandados

Investigar de varias fuentes y realizar su propio informe sobre cuáles son los 5 lenguajes de programación más demandados en el mercado laboral argentino. Indicar año de creación, creadores, principal campo de aplicación, a qué nivel corresponde según la abstracción del procesador. Debe incluir un comentario final, que incluya su apreciación, sobre cuál cree que será el campo de aplicación de la programación en los próximos 10 años.

1.1. Metodología de investigación

Para realizar este informe se consultaron diversas fuentes especializadas en tecnología y empleo IT, incluyendo portales de búsqueda laboral argentinos, índices internacionales como TIOBE y PYPL, y artículos de medios especializados en tecnología. Se analizaron las tendencias de ofertas laborales en Argentina durante 2024-2025 y se identificaron los lenguajes con mayor demanda en el mercado local.

1.2. Los 5 lenguajes más demandados en Argentina

1.2.1. 1. Python

- **Año de creación:** 1991
- **Creador:** Guido van Rossum (Países Bajos)
- **Nivel de abstracción:** Alto nivel (muy alejado del lenguaje máquina)
- **Campo de aplicación principal:** Ciencia de datos, inteligencia artificial, machine learning, automatización, desarrollo web backend, análisis de datos
- **Por qué es demandado:** Sintaxis clara y legible, versatilidad, amplio ecosistema de bibliotecas (TensorFlow, Pandas, NumPy), ideal para IA y análisis de datos

1.2.2. 2. JavaScript

- **Año de creación:** 1995
- **Creador:** Brendan Eich (Netscape Communications)
- **Nivel de abstracción:** Alto nivel
- **Campo de aplicación principal:** Desarrollo web frontend (interfaces interactivas), desarrollo backend (Node.js), aplicaciones móviles (React Native), aplicaciones de escritorio

- **Por qué es demandado:** Lenguaje esencial para desarrollo web, se ejecuta en navegadores, frameworks populares (React, Angular, Vue), permite desarrollo Full-Stack

1.2.3. 3. Java

- **Año de creación:** 1995
- **Creador:** James Gosling (Sun Microsystems, ahora Oracle)
- **Nivel de abstracción:** Alto nivel
- **Campo de aplicación principal:** Aplicaciones empresariales, sistemas bancarios, aplicaciones Android, sistemas de gran escala, backend de aplicaciones web
- **Por qué es demandado:** Portabilidad multiplataforma (“write once, run anywhere”), robustez, fuerte presencia en sector financiero y empresas grandes, gran cantidad de sistemas legacy

1.2.4. 4. C# (C Sharp)

- **Año de creación:** 2000
- **Creador:** Microsoft (Anders Hejlsberg)
- **Nivel de abstracción:** Alto nivel
- **Campo de aplicación principal:** Desarrollo de aplicaciones Windows, desarrollo de videojuegos (Unity), aplicaciones empresariales con .NET, desarrollo web con ASP.NET
- **Por qué es demandado:** Integración con ecosistema Microsoft, usado en Unity (motor de videojuegos muy popular), aplicaciones empresariales, desarrollo multiplataforma con .NET Core

1.2.5. 5. SQL (Structured Query Language)

- **Año de creación:** 1974
- **Creadores:** Donald D. Chamberlin y Raymond F. Boyce (IBM)
- **Nivel de abstracción:** Lenguaje de consulta de alto nivel (no es de programación general, sino especializado en bases de datos)
- **Campo de aplicación principal:** Gestión y consulta de bases de datos relacionales, análisis de datos, reporting, administración de datos

- **Por qué es demandado:** Indispensable para trabajar con bases de datos, usado en prácticamente todas las aplicaciones que manejan datos, complementa cualquier otro lenguaje de programación

1.3. Comparación de niveles de abstracción

Todos los lenguajes mencionados son de **alto nivel**, lo que significa que están muy alejados del lenguaje máquina y cercanos al lenguaje humano. Esto los hace más fáciles de aprender y escribir, aunque requieren compiladores o intérpretes para ejecutarse. SQL es un caso especial: es un lenguaje de consulta, no de programación general.

1.4. Apreciación personal: El futuro de la programación

Considero que en los próximos 10 años (2026-2036), el campo de aplicación de la programación estará dominado por tres áreas principales:

1. Inteligencia Artificial y Automatización: La programación se enfocará cada vez más en desarrollar sistemas inteligentes que automatizan tareas complejas. Python seguirá siendo dominante en este campo, pero surgirán nuevos lenguajes especializados en IA. Los programadores no solo escribirán código, sino que entrenarán y supervisarán modelos de IA.

2. Internet de las Cosas (IoT) y Computación en la Nube: Con miles de millones de dispositivos conectados (desde electrodomésticos hasta sensores industriales), la programación se centrará en gestionar esta red masiva de dispositivos, procesar datos en tiempo real y garantizar la seguridad. Lenguajes como JavaScript, Python y Go ganarán terreno.

3. Realidad Aumentada/Virtual y Metaverso: El desarrollo de experiencias inmersivas requerirá programadores especializados en gráficos 3D, física simulada e interacción en tiempo real. C# (Unity), C++ (Unreal Engine) y nuevos frameworks de realidad extendida serán cruciales.

Además, creo que surgirá una nueva necesidad: **programación ética y responsable**. Con la IA tomando decisiones que afectan vidas humanas, los programadores deberán considerar aspectos éticos, sesgos algorítmicos y transparencia en sus sistemas.

En Argentina específicamente, veo un gran potencial en el desarrollo de soluciones tecnológicas para Latinoamérica, aprovechando el talento local para crear software adaptado a las necesidades regionales en áreas como fintech, agtech (tecnología agrícola) y healthtech (tecnología médica).

El futuro de la programación no será solo escribir código más eficiente, sino crear sistemas que mejoren genuinamente la calidad de vida de las personas, manteniendo siempre un equilibrio entre innovación y responsabilidad.

[↑ Volver al índice](#)

2. ¿Qué es una computadora?

Una computadora es un dispositivo electrónico, utilizado para procesar información y obtener resultados, capaz de ejecutar cálculos y tomar decisiones a velocidades millones o cientos de millones más rápidas que puedan hacerlo los seres humanos. En el sentido más simple una computadora es “un dispositivo” para realizar cálculos o computar. El término sistema de computadora o simplemente computadora se utiliza para enfatizar que, en realidad, son dos partes distintas: hardware y software. El hardware es la computadora en sí misma. El software es el conjunto de programas que indican a la computadora las tareas que debe realizar. Las computadoras procesan datos bajo el control de un conjunto de instrucciones denominadas programas de computadora. Estos programas controlan y dirigen a la computadora para que realice un conjunto de acciones (instrucciones) especificadas por personas especializadas, llamadas programadores de computadoras.

[↑ Volver al índice](#)

3. ¿Qué es la lógica? Ejemplos de la vida real donde aplique la lógica

El término «lógica» proviene de la palabra griega *logos*, que a veces se traduce como «sentencia», «discurso», «regla» y «razón». Podríamos definirla como el estudio de los principios del razonamiento correcto. Analiza cómo llegamos a conclusiones correctas a partir de ciertas premisas.

Ejemplo de la vida real: "Si estudio, apruebo."

[↑ Volver al índice](#)

4. ¿Un Programa tiene relación con la lógica? Justificar

Sí, están relacionados. En un programa se evalúan condiciones y se toman decisiones todo el tiempo, las cuales están basadas en la lógica.

[↑ Volver al índice](#)

5. ¿Qué es un lenguaje? Dar ejemplos reales

Un lenguaje es un sistema de comunicación formado por signos, símbolos o reglas que permite expresar ideas, pensamientos, información o instrucciones entre individuos o sistemas.

Ejemplos:

1. **Lenguaje natural (humano):** Son los que usamos diariamente para comunicarnos.
2. **Lenguaje matemático:** Utiliza símbolos y números para representar ideas.
3. **Lenguaje de programación:** Permite comunicarnos con una computadora.
4. **Lenguaje corporal:** Se comunica mediante gestos y expresiones.

Por lo tanto, un lenguaje es cualquier sistema estructurado que permite comunicar información. Puede ser:

- Verbal (español)
- Simbólico (matemática)
- Técnico (programación)
- No verbal (gestos)

[↑ Volver al índice](#)

6. ¿Cómo se relaciona la matemática con la informática?

La informática se apoya directamente en la matemática: no solo como herramienta, sino como fundamento teórico. Gran parte de lo que hace un programa —calcular, decidir, optimizar— está basado en estructuras y principios matemáticos.

[↑ Volver al índice](#)

7. ¿Qué operadores aritméticos y lógicos conoce?

Los operadores son símbolos que permiten realizar operaciones sobre valores y variables. A continuación se presentan los operadores aritméticos y lógicos más comunes en programación:

7.1. Operadores Aritméticos

Los operadores aritméticos se utilizan para realizar operaciones matemáticas básicas.

Cuadro 1: Operadores Aritméticos

Operador	Nombre	Descripción	Ejemplo
+	Suma	Suma dos valores	$5 + 3 = 8$
-	Resta	Resta el segundo valor del primero	$5 - 3 = 2$
*	Multiplicación	Multiplica dos valores	$5 * 3 = 15$
/	División	Divide el primer valor por el segundo	$15 / 3 = 5$
%	Módulo	Devuelve el resto de la división	$5 \% 3 = 2$
** o ^	Potencia	Eleva un número a una potencia	$5 ** 2 = 25$
//	División entera	División que retorna solo la parte entera	$5 // 2 = 2$

7.2. Operadores Lógicos

Los operadores lógicos se utilizan para realizar operaciones booleanas y evaluar condiciones.

Cuadro 2: Operadores Lógicos

Operador	Nombre	Descripción	Ejemplo
AND o &&	Y lógico	Verdadero si ambas condiciones son verdaderas	$(5 > 3) \text{ AND } (2 < 4)$
OR o	O lógico	Verdadero si al menos una condición es verdadera	$(5 > 3) \text{ OR } (2 > 4)$
NOT o !	Negación	Invierte el valor de verdad	$\text{NOT } (5 > 3)$

7.3. Operadores Relacionales

Además, existen operadores relacionales que se utilizan para comparar valores:

Cuadro 3: Operadores Relacionales

Operador	Nombre	Descripción	Ejemplo
==	Igualdad	Verdadero si ambos valores son iguales	5 == 5
!= o <>	Diferente	Verdadero si los valores son diferentes	5 != 3
>	Mayor que	Verdadero si el primero es mayor	5 > 3
<	Menor que	Verdadero si el primero es menor	3 < 5
>=	Mayor o igual	Verdadero si es mayor o igual	5 >= 5
<=	Menor o igual	Verdadero si es menor o igual	3 <= 5

[↑ Volver al índice](#)

8. ¿Sabe de qué hablamos con la jerarquía de las operaciones?

La **jerarquía de operaciones** (también llamada prioridad o precedencia de operadores) es el conjunto de reglas que determinan el orden en que se deben realizar las operaciones en una expresión matemática o de programación.

8.1. Reglas de jerarquía

Las operaciones se evalúan en el siguiente orden de mayor a menor prioridad:

1. **Paréntesis ()**: Las expresiones dentro de paréntesis se evalúan primero. Si hay paréntesis anidados (unos dentro de otros), se evalúan desde los más internos hacia los más externos.
2. **Exponenciación $\wedge$ o $**$** : Potencias y raíces.
3. **Multiplicación $*$ y División $/$** : Tienen la misma prioridad entre sí.
4. **Suma $+$ y Resta $-$** : Tienen la misma prioridad entre sí, y son las de menor precedencia.

8.2. Regla de asociatividad

Cuando en una expresión coinciden varios operadores con la **misma prioridad**, se evalúan **de izquierda a derecha**.

Ejemplo:

$$8 \div 4 \times 2 = (8 \div 4) \times 2 = 2 \times 2 = 4$$

[↑ Volver al índice](#)

9. ¿Cómo se puede alterar la jerarquía natural de las operaciones?

La jerarquía natural de las operaciones se puede alterar utilizando **paréntesis**. Los paréntesis tienen la máxima prioridad en cualquier expresión, por lo que todo lo que esté dentro de ellos se evalúa primero, sin importar qué operadores contenga.

Ejemplo sin paréntesis:

$$5 + 3 \times 2 = 5 + 6 = 11$$

Aquí la multiplicación se ejecuta primero (jerarquía natural).

Ejemplo con paréntesis:

$$(5 + 3) \times 2 = 8 \times 2 = 16$$

Aquí los paréntesis fuerzan que la suma se ejecute primero, alterando la jerarquía natural.

Regla práctica: Cuando queremos que una operación de menor prioridad (como suma o resta) se ejecute antes que una de mayor prioridad (como multiplicación o división), usamos paréntesis para indicarlo explícitamente.

[↑ Volver al índice](#)

10. ¿Qué es un algoritmo?

Un algoritmo es un método para resolver un problema mediante una serie de pasos precisos, definidos y finitos.

- **Preciso:** Indica el orden de realización en cada paso.
- **Definido:** Si se sigue dos veces, se obtiene el mismo resultado cada vez.
- **Finito:** Tiene fin; un número determinado de pasos.

Un algoritmo debe producir un resultado en un tiempo finito. Los métodos que utilizan algoritmos se denominan métodos algorítmicos, en oposición a los métodos que implican algún juicio o interpretación que se denominan métodos heurísticos. Los algoritmos se pueden expresar por fórmulas, diagramas de flujo y pseudocódigos.

[↑ Volver al índice](#)

11. ¿En dónde encontramos algoritmos en la vida real?

Ejemplos de algoritmos en la vida real pueden ser: instrucciones para armar una bicicleta, hacer una receta de cocina, obtener el máximo común divisor de dos números, etc.

[↑ Volver al índice](#)

12. Realizar el algoritmo de despertar y levantarse diariamente

A continuación se presenta el algoritmo de la rutina diaria de despertar y levantarse.

```
1 Algoritmo DespertarDiario
2
3     Definir dia Como Entero
4
5     // Este ciclo representa que se repite cada día
6     Para dia <- 1 Hasta 7 Con Paso 1 Hacer
7         Escribir "=== DÍA ", dia, " ==="
8
9         Escribir "1. Suena la alarma"
10
11        Escribir "2. Apagar la alarma"
12
13        Escribir "3. Abrir los ojos"
14
15        Escribir "4. Estirar el cuerpo"
16
17        Escribir "5. Levantarse de la cama"
18
19        Escribir ""
20    FinPara
21
22    Escribir "=== FIN DE LA SEMANA ==="
23
24 FinAlgoritmo
```

Listing 1: Algoritmo de despertar y levantarse diariamente (PSeInt)

12.1. Descripción del algoritmo

El algoritmo refleja la naturaleza **repetitiva** de la rutina matutina utilizando una estructura Para:

- **Estructura de repetición:** El ciclo Para simula que la rutina se ejecuta cada día (del día 1 al 7, representando una semana)
- **Pasos secuenciales:** Dentro de cada día, los pasos se ejecutan en orden:
 1. Suena la alarma
 2. Apagar la alarma

3. Abrir los ojos
4. Estirar el cuerpo
5. Levantarse de la cama

[↑ Volver al índice](#)

13. ¿Qué es un programa? ¿Qué es un lenguaje de programación? ¿Y qué tarea es la que hace un programador?

¿Qué es un programa?

Definimos programa como un conjunto de instrucciones y sus tipos. Un programa de computadora es un conjunto de instrucciones -órdenes dadas a la máquina- que producirán la ejecución de una determinada tarea. En esencia, un programa es un medio para conseguir un fin. El fin será probablemente definido como la información necesaria para solucionar un problema.

¿Qué es un lenguaje de programación?

Los lenguajes de programación son todos los símbolos, caracteres y reglas de uso que permiten a las personas comunicarse con las computadoras. Un lenguaje de programación es un sistema estructurado y diseñado principalmente para que las computadoras se entiendan entre sí y con nosotros, los humanos. Contiene un conjunto de acciones consecutivas que el ordenador debe ejecutar.

¿Qué tarea hace un programador?

Un programador es la persona encargada de diseñar, escribir, probar y mantener programas de computadora. Sus principales tareas incluyen:

- **Analizar problemas:** Entender qué problema necesita resolverse y cómo la computadora puede ayudar a solucionarlo.
- **Diseñar soluciones:** Crear algoritmos y planificar la estructura del programa antes de escribir código.
- **Escribir código:** Traducir los algoritmos a un lenguaje de programación específico que la computadora pueda ejecutar.
- **Probar y depurar:** Ejecutar el programa para encontrar y corregir errores (bugs).
- **Mantener y actualizar:** Mejorar programas existentes, agregar nuevas funcionalidades o adaptarlos a nuevas necesidades.

En resumen, el programador actúa como traductor entre las necesidades humanas y el lenguaje que entiende la computadora, transformando problemas del mundo real en soluciones computacionales.

[↑ Volver al índice](#)

14. ¿A qué se denomina un programador Full-Stack?

Un programador **Full-Stack** (o de “pila completa”) es un desarrollador que tiene conocimientos y habilidades tanto en el **frontend** (la parte visible de una aplicación) como en el **backend** (el servidor y la lógica de negocio).

Frontend: Es lo que el usuario ve e interactúa directamente. Incluye:

- Diseño de interfaces (HTML, CSS)
- Interactividad y comportamiento (JavaScript)
- Experiencia de usuario (UX/UI)

Backend: Es la parte del servidor que procesa datos y lógica. Incluye:

- Servidores y APIs
- Bases de datos
- Lógica de negocio y seguridad
- Lenguajes como Python, Java, Node.js, PHP

Un programador Full-Stack puede trabajar en todas las capas de una aplicación web, desde el diseño de la interfaz hasta la gestión de bases de datos, lo que le permite desarrollar aplicaciones completas de forma independiente. Esta versatilidad lo hace muy valioso en equipos pequeños o startups, aunque en proyectos grandes suele especializarse en alguna de las dos áreas.

[↑ Volver al índice](#)

15. ¿Qué fases en la “resolución de problemas” conoce? Mencionalas, seleccione la que le parezca más le llame la atención explicando su importancia y sumando un ejemplo

El proceso de resolución de un problema con una computadora conduce a la escritura de un programa y a su ejecución en la misma. Aunque el proceso de diseñar programas es, esencialmente, un proceso creativo, se puede considerar una serie de fases o pasos comunes que generalmente deben seguir todos los programadores.

15.1. Fases de la resolución de problemas

Las fases principales son:

1. **Análisis del problema:** Se analiza el problema teniendo presente la especificación de los requisitos dados por quien encarga el programa.
2. **Diseño del algoritmo:** Una vez analizado el problema, se diseña una solución que conducirá a un algoritmo que lo resuelva.
3. **Codificación (implementación):** La solución se escribe en la sintaxis del lenguaje de alto nivel elegido (por ejemplo, Python, Java o C) y se obtiene un programa fuente.
4. **Compilación y ejecución:** El programa se traduce a lenguaje máquina y se ejecuta.
5. **Verificación:** Se comprueba rigurosamente que el programa funcione correctamente.
6. **Depuración:** Se eliminan todos los errores (denominados “bugs”) que puedan aparecer.
7. **Mantenimiento:** El programa se actualiza y modifica cada vez que sea necesario, de modo que se cumplan todas las necesidades de cambio de sus usuarios.
8. **Documentación:** Escritura de las diferentes fases del ciclo de vida del software, manuales de usuario y de referencia, así como normas para el mantenimiento.

15.2. Fase que más me interesa: Análisis y Diseño

Las fases que considero más importantes son las **dos primeras**: el **análisis** y el **diseño del algoritmo**. Estas etapas son donde realmente se resuelve el problema de forma conceptual

y lógica.

Una vez que se tiene claro *qué* hay que hacer y *cómo* hacerlo (el algoritmo), la codificación es relativamente sencilla: se trata simplemente de traducir esa solución a la sintaxis del lenguaje de programación elegido. Es decir, la parte difícil no es programar, sino **pensar la solución**.

Si el análisis y diseño están bien hechos, el código prácticamente “se escribe solo”. En cambio, si estas fases se hacen mal, por más que se domine la sintaxis del lenguaje, el programa no resolverá correctamente el problema.

15.3. Ejemplo práctico

Problema: Calcular el promedio de calificaciones de un estudiante y determinar si aprobó (promedio ≥ 6).

1. Análisis del problema:

- *Entrada:* Lista de calificaciones del estudiante
- *Proceso:* Sumar todas las calificaciones y dividir por la cantidad
- *Salida:* Promedio y mensaje de “Aprobado” o “Desaprobado”
- *Condición:* Si promedio ≥ 6 entonces aprobó

2. Diseño del algoritmo:

1. Leer la cantidad de calificaciones (n)
2. Inicializar suma en 0
3. Para cada calificación:
 - Leer la calificación
 - Sumarla al total
4. Calcular promedio = suma / n
5. Si promedio ≥ 6 :
 - Mostrar “Aprobado”
6. Si no:
 - Mostrar “Desaprobado”
7. Mostrar el promedio

3. Codificación (en este punto, ya solo traducimos el algoritmo):

```
1 # La logica ya esta resuelta, ahora solo escribimos sintaxis
2 n = int(input("Cantidad de calificaciones: "))
3 suma = 0
4
5 for i in range(n):
6     calificacion = float(input(f"Calificacion {i+1}: "))
7     suma += calificacion
8
9 promedio = suma / n
10
11 if promedio >= 6:
12     print("Aprobado")
13 else:
14     print("Desaprobado")
15
16 print(f"Promedio: {promedio}")
```

Listing 2: Ejemplo en Python

Como se puede observar, una vez que se completaron las fases de análisis y diseño, la codificación fue directa: simplemente se tradujo la lógica ya pensada a la sintaxis de Python. El verdadero trabajo intelectual ocurrió en las dos primeras fases.

[↑ Volver al índice](#)

16. ¿Qué es una variable y qué es una constante?

Constante:

Una constante es un espacio en memoria cuyo valor **no puede cambiar** durante la ejecución del programa. Se define una vez y permanece fija.

Ejemplo: $\pi = 3.1415926$, $\text{IVA} = 0.21$

Variable:

Una variable es un espacio en memoria identificado por un nombre simbólico, cuyo valor **puede cambiar** durante la ejecución del programa.

Ejemplo: $\text{edad} = 20$, luego $\text{edad} = 21$

Diferencia clave: Las constantes mantienen su valor fijo, mientras que las variables pueden modificarse cuantas veces sea necesario durante la ejecución del programa.

[↑ Volver al índice](#)

17. ¿Qué es Software y qué es Hardware?

Hardware:

El hardware es la parte **física y tangible** de una computadora. Son todos los componentes que se pueden tocar.

Ejemplos: Procesador (CPU), memoria RAM, disco duro, teclado, mouse, monitor, placa madre.

Software:

El software es la parte **lógica e intangible** de una computadora. Son los programas e instrucciones que le indican al hardware qué hacer.

Ejemplos: Sistema operativo (Windows, Linux), navegadores web, Microsoft Word, videojuegos, aplicaciones móviles.

Relación: El hardware es el “cuerpo” de la computadora y el software es el “cerebro”. Ambos son necesarios: el hardware sin software no puede hacer nada útil, y el software necesita hardware para ejecutarse.

[↑ Volver al índice](#)

18. ¿Qué es la memoria RAM y la memoria ROM?

Memoria RAM (Random Access Memory - Memoria de Acceso Aleatorio):

Es una memoria **volátil** y **temporal**. Almacena datos y programas que se están ejecutando en ese momento. Cuando se apaga la computadora, toda la información en la RAM se pierde.

Características: Lectura y escritura rápida, se borra al apagar el equipo, mayor capacidad (4GB, 8GB, 16GB, etc.).

Memoria ROM (Read Only Memory - Memoria de Solo Lectura):

Es una memoria **no volátil** y **permanente**. Almacena instrucciones básicas necesarias para iniciar la computadora (BIOS). Su contenido no se borra al apagar el equipo.

Características: Solo lectura (no se puede modificar fácilmente), mantiene información sin energía, menor capacidad.

Diferencia principal: La RAM es rápida pero temporal (se borra sin energía), mientras que la ROM es permanente pero más lenta (conserva la información sin energía).

[↑ Volver al índice](#)

19. ¿Qué son los dispositivos de entrada y salida? Dé 5 ejemplos de cada uno

Dispositivos de Entrada:

Son aquellos que permiten **ingresar información** desde el exterior hacia la computadora. El usuario envía datos al sistema.

5 ejemplos:

1. Teclado
2. Mouse (ratón)
3. Micrófono
4. Escáner
5. Cámara web

Dispositivos de Salida:

Son aquellos que permiten **mostrar o extraer información** desde la computadora hacia el exterior. El sistema envía datos al usuario.

5 ejemplos:

1. Monitor (pantalla)
2. Impresora
3. Parlantes (altavoces)
4. Auriculares
5. Proyector

Nota: Existen también dispositivos de **entrada/salida** (mixtos) como pantallas táctiles, impresoras multifunción o discos duros, que pueden tanto recibir como enviar información.

[↑ Volver al índice](#)

20. Nombrar las unidades principales de un procesador

Un procesador (CPU - Unidad Central de Procesamiento) está compuesto por tres unidades principales:

1. **Unidad de Control (UC):** Se encarga de controlar y coordinar todas las operaciones del procesador. Interpreta las instrucciones del programa y genera las señales necesarias para ejecutarlas.
2. **Unidad Aritmético-Lógica (ALU):** Realiza todas las operaciones aritméticas (suma, resta, multiplicación, división) y lógicas (comparaciones, AND, OR, NOT).
3. **Registros:** Son pequeñas unidades de memoria de alta velocidad dentro del procesador que almacenan temporalmente datos e instrucciones que se están procesando en ese momento.

Estas tres unidades trabajan en conjunto para ejecutar las instrucciones de los programas: la UC coordina, la ALU calcula, y los registros almacenan datos temporales durante el procesamiento.

[↑ Volver al índice](#)